

General Review of Hash-Based Signatures

Halil İbrahim Kaplan¹[0009–0006–0120–6512]

Cryptanalysis Laboratory, TÜBİTAK BİLGEM, Kocaeli, Turkey
halil.kaplan@tubitak.gov.tr

Abstract. The advent of quantum computing threatens the security assumptions underpinning classical public-key cryptographic algorithms such as RSA and ECC. As a response, the cryptographic community has focused on developing quantum-resistant alternatives, with hash-based signature schemes emerging as a compelling option due to their reliance on well-understood hash functions rather than number-theoretic hardness assumptions. This paper presents a comprehensive review of hash-based signature schemes, including Lamport, WOTS, XMSS, $XMSS^{MT}$, and SPHINCS+, examining their structural design, key generation, signing, and verification processes. Emphasis is placed on their classification as stateful and stateless schemes, as well as their practical integration using Merkle trees and address structures. Furthermore, the paper analyzes several notable cryptanalytic attacks—such as intermediate value guessing, Antonov’s attack, multi-target attacks, and fault injection strategies—that pose risks to these constructions. By discussing both their strengths and vulnerabilities, this work highlights the viability of hash-based signatures as secure and efficient candidates for post-quantum digital signatures.

Keywords: Post-quantum cryptography · Hash-based signatures · XMSS · SPHINCS+ · Cryptanalysis

1 Introduction

Hash-based signature (HBS) schemes have been known since the late 1970s, with early contributions by Lamport and Merkle. However, for many years, they remained theoretical constructs without widespread practical use, primarily due to their large signature sizes and, in the case of stateful schemes, complex key management requirements. This perception has shifted significantly with the emergence of post-quantum cryptography (PQC), which aims to develop cryptographic primitives that are resistant to quantum attacks.

The threat posed by quantum computers—particularly Shor’s algorithm, which can break RSA and elliptic curve cryptography (ECC)—has led to renewed interest in HBS schemes. Unlike number-theoretic approaches, HBS algorithms rely solely on the security of cryptographic hash functions, which are believed to remain secure even in the presence of quantum adversaries.

This growing interest has been formalized through standardization efforts. The Internet Engineering Task Force (IETF) published RFC 8391 for the eX-

tended Merkle Signature Scheme (XMSS) in 2018, followed by the Leighton-Micali Signature (LMS) scheme. In parallel, the National Institute of Standards and Technology (NIST) provided parameter recommendations and, in 2022, selected the stateless scheme SPHINCS+ as a finalist in its post-quantum cryptography standardization project. These developments were also supported by national agencies such as ANSSI (France) and BSI (Germany), which incorporated XMSS and LMS into their respective cryptographic guidelines.

HBS schemes have since seen increasing deployment in embedded systems, firmware signing, and secure boot mechanisms—scenarios where post-quantum resilience and long-term security are critical. However, the limitations of stateful schemes like XMSS and LMS, particularly the bounded number of signatures per key, restrict their usage to environments where state can be reliably managed. Stateless alternatives such as SPHINCS+ offer more flexibility at the cost of larger signatures and slower performance.

This paper presents a comprehensive study of modern HBS schemes, focusing on the structural foundations, key and signature generation algorithms, and the design differences between XMSS and SPHINCS+. In addition to the algorithmic descriptions, we analyze the trade-offs between stateful and stateless constructions and investigate critical components such as address structures, Merkle trees, and Winternitz one-time signatures (WOTS). Furthermore, we explore known cryptanalytic approaches targeting HBS constructions, with an emphasis on multi-target attacks, fault injection strategies, and signature forgeries enabled through checksum manipulation or hash chain tampering.

The purpose of this study is to bridge theoretical constructs with practical implications, offering a clear understanding of how hash-based signatures works.

2 Basic Structures

In this section, the basic structures needed to understand hash-based signature algorithms will be explained.

2.1 Hash Functions

Cryptographic hash functions are functions that convert input data of any length into a fixed-length digest value. In order for these functions to be used securely, they must satisfy certain fundamental properties. These properties are as follows:

1. **Collision Resistance:** The probability that two different inputs produce the same hash value must be very low. In other words, the condition $h(x) = h(y)$ for $x \neq y$ should be extremely rare. This property is one of the cornerstones of hash function security, and finding collisions should be computationally infeasible.
2. **Pre-image Resistance:** It must be difficult to find the input that produces a given hash value. That is, given any hash output $h(x)$, it should be computationally hard to find the corresponding input x .

- 3. Second Pre-image Resistance:** Given a specific input and its hash value, it should be difficult to find a different input that produces the same hash value. In other words, given $h(x)$, it should be computationally hard to find a different $y \neq x$ such that $h(y) = h(x)$.

Attack complexities of n -bit Hash function is given in Table 1

Table 1: Attack Complexities on an n -bit Hash

Attack	Classical Work	Quantum Work
Preimage	2^n	$2^{n/2}$
Second-Preimage	2^n	$2^{n/2}$
Collision	$2^{n/2}$	$2^{n/3}$ (Brassard-Høyer-Tapp [3])

2.2 Lamport One-Time Signature

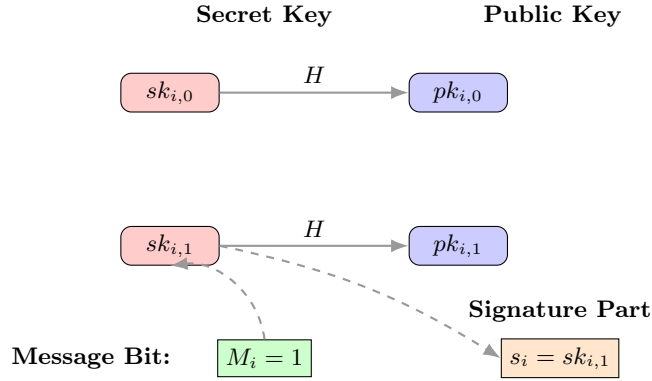


Fig. 1: Lamport Signature Scheme

The Lamport signature scheme [8] is the first known hash-based signature algorithm. This scheme generates signatures that can only be used once, hence requiring a different key pair for each message. The steps of the scheme are as follows:

Lamport Key Generation Let us assume that a 256-bit message will be signed. In the Lamport algorithm, each bit of the signature requires 2 keys, resulting in a total of 512 public and private keys. The private keys are generated randomly and grouped as follows:

$$\text{sk}_0 = \text{sk}_{10}, \text{sk}_{20}, \dots, \text{sk}_{2560}$$

$$\text{sk}_1 = \text{sk}_{11}, \text{sk}_{21}, \dots, \text{sk}_{2561}$$

The public keys are obtained by applying a hash function to each private key:

$$\text{pk}_0 = H(\text{sk}_{10}), H(\text{sk}_{20}), \dots, H(\text{sk}_{2560})$$

$$\text{pk}_1 = H(\text{sk}_{11}), H(\text{sk}_{21}), \dots, H(\text{sk}_{2561})$$

Lamport Signature Each bit of the message is examined. If the bit is 0, the corresponding part of the signature is taken from the sk_0 group; if the bit is 1, it is taken from the sk_1 group.

For example, suppose the first three bits of our message are "001". Then, the first three parts of the signature would be:

$$\text{sk}_{10} \parallel \text{sk}_{20} \parallel \text{sk}_{31}$$

Signature Verification The verifier examines each bit of the message one by one. If the bit value is 0, the hash of the corresponding part of the signature is computed and compared with the corresponding part of pk_0 . If the bit is 1, it is compared with the corresponding part of pk_1 .

This process is repeated for all bits. If all comparisons are equal, then the signature is successfully verified.

2.3 WOTS and WOTS+

The WOTS (Winternitz One-Time Signature) algorithm [4], like the Lamport signature algorithm, is a one-time signature scheme. While in the Lamport algorithm each bit of the message is signed individually, in the WOTS scheme the message is divided into chunks of w bits, and each chunk is signed instead of individual bits.

As a result, the signature size becomes w times smaller compared to the Lamport scheme. However, a w -bit message chunk can take 2^w different values, and a private key is required for each possible value.

This problem is also addressed using hash functions.

WOTS Key generation Let us assume that the message is divided into N parts, each consisting of l bits. In this case, N secret keys $\text{sk}_0, \text{sk}_1, \dots, \text{sk}_N$ are randomly generated. The 2^l possible values that each part can take are obtained by applying a hash function repeatedly to the randomly generated keys.

In Figure 2, the secret keys are shown. The public key is obtained by applying the hash function 2^l times to each secret key (i.e., the 2^l -th hash).

For example, if the value of the first message chunk is 0, then sk_0 is used in the signature. If the value is $2^l - 1$, then the signature value is $h^{2^l-1}(\text{sk}_0)$.

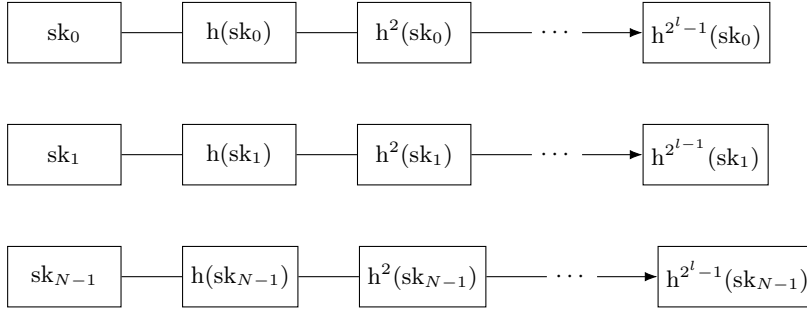


Fig. 2: WOTS key generation

WOTS Signature For each part of the message, the appropriate number of hash iterations is applied to the corresponding secret key. These hashed values are concatenated to form the complete signature.

For example, suppose the first three chunks of the message are:

$$001\|010\|111$$

Then the first three parts of the signature would be:

$$h(sk_0)\|h^2(sk_1)\|h^7(sk_2)$$

WOTS Signature Verification The verifier examines each message part individually. By checking the numerical value of each chunk, the verifier determines how many hash iterations are needed to reach the public key. It then hashes the corresponding part of the signature that many times. The result is compared with the known public key. If all values match, the signature is considered valid.

WOTS+ [7] Introduced as an improved variant of WOTS. The primary difference is that WOTS+ introduces a randomized bitmasking technique in its chaining function. Specifically, before each hash computation in the signature generation and verification process, WOTS+ XORs a randomly generated bit-mask with the input. This masking mitigates multi-target attacks and improves the scheme's resistance to attacks that exploit structural properties of the hash function.

2.4 Merkle Trees and Multi-Signatures

Lamport and WOTS schemes are one-time signature algorithms, meaning they allow signing only a single message with generated keys. However, in practical scenarios, signature algorithms are expected to support multiple signatures. To convert one-time signature schemes into algorithms that can sign multiple messages, **Merkle trees** are used.

As an example, the WOTS scheme can be used together with a Merkle tree. Figure 3 shows the sketch of merkle tree used with WOTS. The bottom leaves of the Merkle tree are the hash values of the WOTS public keys. Using a Merkle tree of height h , a signature scheme capable of signing 2^h messages can be constructed.

In addition to increasing signature capacity, Merkle trees also have the advantage of **reducing public key size**. Normally, a WOTS public key consists of multiple components—one for each message chunk—but when used with a Merkle tree, only **one public key** is required: the root of the Merkle tree. To verify a signature, the verifier must access certain intermediate nodes of the Merkle tree in order to compute the root. These intermediate values can be included as part of the signature, thus enabling successful verification.

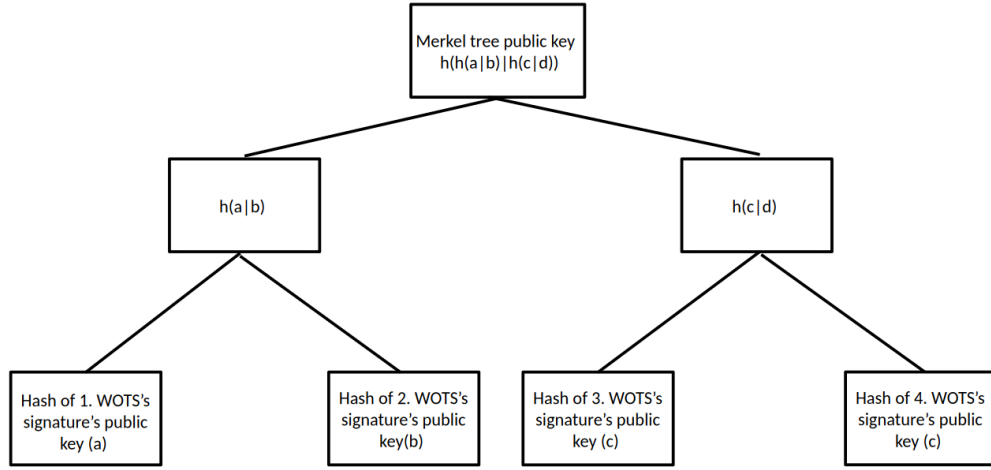


Fig. 3: Merkle Tree

2.5 L-tree and Hashing public key

When combining one-time signature schemes with a Merkle tree, the public key of the one-time signature scheme is selected as the bottom leaf of the Merkle tree. However, in the WOTS algorithm, a single signature includes not one but multiple public key components—one for each message part.

To resolve this issue, two different approaches are employed:

1. **L-tree (used in XMSS):** The L-tree is a specialized Merkle tree used to compress multiple public key components into a single public key. It repeatedly hashes pairs of public key elements until only one remains.

2. **Hashing (used in SPHINCS+):** In this method, all public key components are concatenated, and a single hash value is computed over the entire concatenation. This resulting hash serves as the single public key for Merkle tree integration.

2.6 Base- w Representation

In hash-based signature schemes such as WOTS and XMSS, the message digest is first transformed into a base- w representation before signing. This **base- w representation** is a critical component of the signing process and directly influences the structure and security of the signature.

The main purposes of using base- w representation are to:

- reduce the size of the signature compared to bit-wise representations (as in the Lamport scheme),
- represent the message digest in a form suitable for the Winternitz one-time signature construction,
- control the number of hash iterations per key chain in WOTS+,
- and balance the trade-off between performance (speed) and signature length through the choice of w .

Let the output of the hash function be n bits. The number of base- w digits required to represent this n -bit message is:

$$\ell_1 = \left\lceil \frac{8n}{\log_2 w} \right\rceil$$

Each digit in this representation is an integer in the range $[0, w-1]$, and indicates how many times the corresponding chain of the private key should be hashed to produce the signature component.

Example: Suppose $n = 256$ and $w = 16$. Then:

$$\ell_1 = \left\lceil \frac{8 \cdot 256}{\log_2 16} \right\rceil = \left\lceil \frac{2048}{4} \right\rceil = 512$$

So, the hash output will be split into 512 base-16 digits.

After computing the base- w representation of the message digest, a checksum is computed and appended (as described in the section 2.7), forming the complete message input for WOTS+.

The choice of w is significant:

- A smaller w increases the number of digits ℓ_1 (and thus the signature size) but reduces the number of hash computations.
- A larger w reduces the signature size but requires more hash computations per signature, increasing signing and verification time.

In summary, the base- w representation enables the WOTS+ signature scheme to efficiently encode the message digest while maintaining flexibility across a range of parameter choices.

2.7 Checksum

In hash-based signature algorithms such as WOTS and XMSS, the message to be signed is first encoded into a base- w representation. However, this encoding alone is not sufficient to ensure security against certain attacks. Therefore, a **checksum** value is appended to the base- w representation of the message. This checksum serves to:

- prevent attacks that attempt to alter message chunks by increasing or decreasing hash iterations,
- protect against fault injection attacks that aim to exploit the absence or corruption of the checksum,
- and detect message modifications or signature reuse.

The checksum is calculated by summing the differences between the maximum possible chunk value ($w - 1$) and each base- w digit of the message. If the message is represented by $m_0, m_1, \dots, m_{\ell_1-1}$ in base- w , the checksum c is given by:

$$c = \sum_{i=0}^{\ell_1-1} (w - 1 - m_i)$$

This value is then encoded into base- w as an additional sequence of digits, forming the checksum part of the message. The total number of checksum digits required is ℓ_2 , calculated as:

$$\ell_2 = \left\lceil \frac{\log_2(\ell_1(w - 1))}{\log_2 w} \right\rceil + 1$$

As a result, the full message to be signed by WOTS+ consists of $\ell = \ell_1 + \ell_2$ base- w digits. Each digit corresponds to the number of hash function applications to be performed on a particular chain of the private key.

The checksum plays a crucial role in maintaining the integrity and one-time nature of the WOTS+ signature. Without it, an attacker could transform a valid signature for one message into a valid signature for another message by manipulating the hash chains, which would compromise the existential unforgeability of the scheme.

2.8 FORS

The **FORS algorithm** (Forest of Random Subsets) is a *few-time signature scheme*. Unlike the WOTS algorithm, in FORS, when the message is divided into parts, a secret key is generated for each possible value of every part, and a small Merkle tree is constructed from these keys. The signature consists of:

- The secret key corresponding to the integer value of the message part.
- The necessary authentication path (intermediate nodes) to reach the root of the Merkle tree.

An example FORS signature for the below message is shown in Figure 4.

100 011 011 101 111 000



Hash-based signature schemes consist of either very large Merkle trees or multi-layered Merkle trees in order to increase their signature capacity. Since the same hash function is repeatedly used within the tree and similar operations are performed, **address structures** are used in hash-based signature schemes to:

- Table 2 shows the **OTS address structure** used in the XMSS algorithm [6]. In the example structure, the following fields appear in order from top to bottom:

- Depending on the algorithm and its internal structure, the format of the address field can vary.

Field	Size
layer address	32 bits
tree address	64 bits
type = 0	32 bits
OTS address	32 bits
chain address	32 bits
hash address	32 bits
keyAndMask	32 bits

Table 2: XMSS OTS Address Format

2.10 Stateful and Stateless Hash-Based Signature Schemes

Hash-based signature algorithms can be categorized into two types: **stateful** and **stateless**. The fundamental difference between these two categories lies in the management of state information during the signature generation and verification processes.

Stateful hash-based signature algorithms require the maintenance of state information while generating signatures. These algorithms rely on *one-time keys* that can be used only once per signature operation. Reusing these keys leads to serious security vulnerabilities. A representative example of such an algorithm is XMSS [6].

Stateless hash-based signature algorithms, in contrast, do not require any state information to be stored during the signing process. A prominent example of a stateless algorithm is SPHINCS+ [1].

3 XMSS and $XMSS^{MT}$

In this section, the XMSS/ $XMSS^{MT}$ algorithms, as defined in the documents [5] [6], will be explained using the fundamental structures presented in Section 2. Architecture of the XMSS is given in the Figure 5

3.1 XMSS Key Generation

In Algorithm 1, a pseudo-code representation of the key generation process of the **XMSS** algorithm is provided. First, the **WOTS** private keys are generated. These keys can be created either completely at random or by passing a seed value through a key generation function. If all keys are generated randomly, they must be stored in memory for the signing process. However, since storing all the keys in memory is not realistic, this method is generally not used. Next, certain seed values are generated to be used throughout the algorithm, and the hash values

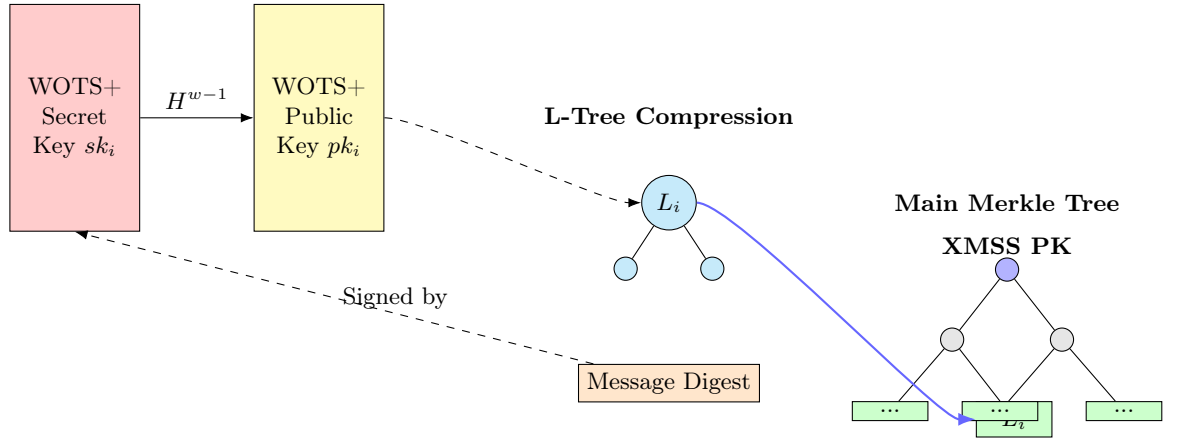


Fig. 5: The architecture of XMSS. A WOTS+ key pair signs a message. Its public key is compressed via an L-Tree into a single leaf L_i . This leaf is part of a large Merkle tree, whose root serves as the overall XMSS public key.

of the WOTS private keys are taken to produce the WOTS public keys. In the pseudo-code, the value `idx` at the beginning of the `SK` section represents the state information.

3.2 XMSS Sign

In Algorithm 2, the pseudo-code of the XMSS signing algorithm is presented. The `treeSig` function signs the message using WOTS and generates the authentication path required to reach the root of the tree. First, the state information is retrieved, then a random value is generated and concatenated with the message, after which a hash is computed from this combination. This hash value is then signed using the `treeSig` algorithm. The final signature consists of the concatenation of the state information, the random value, and the `treeSig` signature, respectively.

3.3 XMSS Verify

In Algorithm 3 and 4, the pseudo-code of the XMSS signature verification is presented. First, the WOTS public key is derived from the message. Then, these WOTS public keys are passed through the `L-tree` to obtain the leaves of the Merkle tree. From the leaves of the Merkle tree, the root is reached to obtain the XMSS public key and perform verification. If the derived public key matches the expected value, the signature is considered valid. The authentication path contained in the signature is used to climb to the root in both the `L-tree` and the Merkle tree.

Algorithm 1 XMSS_keyGen – Generate an XMSS key pair

Input: None**Output:** XMSS private key SK , XMSS public key PK

```

1: // Example initialization for SK-specific contents
2:  $idx \leftarrow 0$ 
3: for  $i = 0$  to  $2^h - 1$  do
4:    $wots\_sk[i] \leftarrow WOTS\_genSK()$ 
5: end for
6: Initialize  $SK\_PRF$  with a uniformly random  $n$ -byte string
7:  $setSK\_PRF(SK, SK\_PRF)$ 
8: // Initialization for common contents
9: Initialize  $SEED$  with a uniformly random  $n$ -byte string
10:  $setSEED(SK, SEED)$ 
11:  $setWOTS\_SK(SK, wots\_sk)$ 
12:  $ADRS \leftarrow toByte(0, 32)$ 
13:  $root \leftarrow treeHash(SK, 0, h, ADRS)$ 
14:  $SK \leftarrow idx \parallel wots\_sk \parallel SK\_PRF \parallel root \parallel SEED$ 
15:  $PK \leftarrow OID \parallel root \parallel SEED$ 
16: return  $SK \parallel PK$ 

```

Algorithm 2 XMSS_sign – Generate an XMSS signature and update the XMSS private key

Input: Message M , XMSS private key SK **Output:** Updated SK , XMSS signature Sig

```

1:  $idx\_sig \leftarrow getIdx(SK)$ 
2:  $setIdx(SK, idx\_sig + 1)$ 
3:  $ADRS \leftarrow toByte(0, 32)$ 
4:  $r \leftarrow PRF(getSK\_PRF(SK), toByte(idx\_sig, 32))$ 
5:  $M' \leftarrow H\_msg(r \parallel getRoot(SK) \parallel toByte(idx\_sig, n), M)$ 
6:  $Sig \leftarrow idx\_sig \parallel r \parallel treeSig(M', SK, idx\_sig, ADRS)$ 
7: return  $SK \parallel Sig$ 

```

Algorithm 3 XMSS_rootFromSig: Compute a root node from a tree signature

Input:

index idx_sig
 WOTS+ signature sig_ots
 authentication path auth
 n -byte message M'
 seed SEED
 address ADRS

Output: n -byte root value $\text{node}[0]$

```

1: ADRS.setType(0)           // Type = OTS hash address
2: ADRS.setOTSAddress(idx_sig)
3: pk_ots ← WOTS_pkFromSig(sig_ots, M', SEED, ADRS)
4: ADRS.setType(1)           // Type = L-tree address
5: ADRS.setLTreeAddress(idx_sig)
6: Initialize node[0] and node[1] as  $n$ -byte arrays
7: node[0] ← ltree(pk_ots, SEED, ADRS)
8: ADRS.setType(2)           // Type = hash tree address
9: ADRS.setTreeIndex(idx_sig)
10: for  $k = 0$  to  $h - 1$  do
11:   ADRS.setTreeHeight(k)
12:   if  $\lfloor \text{idx\_sig} / 2^k \rfloor \bmod 2 = 0$  then
13:     ADRS.setTreeIndex(ADRS.getTreeIndex() / 2)
14:     node[1] ← RAND_HASH(node[0], auth[k], SEED, ADRS)
15:   else
16:     ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2)
17:     node[1] ← RAND_HASH(auth[k], node[0], SEED, ADRS)
18:   end if
19:   node[0] ← node[1]        // Update current node
20: end for
21: return node[0]

```

Algorithm 4 XMSS_verify – Verify an XMSS signature using the corresponding XMSS public key and a message

Input: XMSS signature Sig , message M , XMSS public key PK **Output:** Boolean

```

1: ADRS ← toByte(0, 32)
2: M' ← H_msg(r || getRoot(PK) || toByte(idx_sig, n), M)
3: node ← XMSS_rootFromSig(idx_sig, sig_ots, auth, M', getSEED(PK), ADRS)
4: if node == getRoot(PK) then
5:   return true
6: else
7:   return false
8: end if

```

$XMSS^{MT}$ is the multi-tree version of the XMSS algorithm. While the XMSS algorithm consists of a single large tree, the multi-tree structure comprises many smaller trees, each signing the public key of the next. The advantage of this approach is that signing is faster because it eliminates the need to generate the entire large tree during the signature verification process. In a multi-tree structure, the signature is formed by the combination of the signatures and authentication paths of each layer. Figure 6a shows the multi-tree signature structure. Figure 6b illustrates the overall structure of the multi-tree.



SPHINCS+ [9] is one of the three signature algorithms standardized by NIST. Structurally, it is very similar to XMSS. Fundamentally, there are two main differences. First, the SPHINCS+ algorithm does not use an L-tree structure. Instead, it transforms WOTS public keys into a single public key by applying

a hash function.(see section 2.5) Second, unlike XMSS, which contains only a WOTS signature, the SPHINCS+ algorithm also includes a FORS signature layer underneath the WOTS signature. Since the FORS signature algorithm is not a one-time signature, it allows multiple signatures to be generated from the same tree. For this reason, SPHINCS+ is a stateless algorithm. Before each signing operation, it selects a random index and generates the signature and authentication path from the part of the tree corresponding to that index. Because it does not require securely storing and updating state information, stateless SPHINCS+ offers a significant advantage over XMSS in this regard. However, the FORS layer, which makes SPHINCS+ stateless, adds an additional layer of signature under WOTS, thereby increasing the signature size compared to XMSS. Due to the tree-based structure of hash-based signature algorithms, both algorithms have the same public key size. Figure 9 illustrates the general structure of the SPHINCS+ algorithm.

4.1 SPHINCS+ Parameters

Parameters and their descriptions are given below:

- n : Hash output size in bits. All outputs of the underlying hash functions (e.g., H_{msg} , PRF, and Merkle-node hashes) are exactly n bits long.
- w : Winternitz parameter. Determines the base for WOTS+ chains. Higher w reduces signature length at the cost of more hash computations per chain.
- ℓ : Total number of n -bit chains in each WOTS+ key or signature, given by

$$\ell = \ell_1 + \ell_2, \quad \ell_1 = \left\lceil \frac{8n}{\log_2 w} \right\rceil, \quad \ell_2 = \left\lceil \frac{\log_2(\ell_1(w-1))}{\log_2 w} \right\rceil + 1.$$

- h : Total height of the Merkle hypertree. The overall number of leaves (XMSS key pairs) is 2^h .
- h' : Height of each XMSS subtree. Each subtree contains $2^{h'}$ leaves, and we arrange d such layers to reach height h .
- d : Number of layers in the hypertree, satisfying

$$d = \frac{h}{h'}.$$

- k : Number of parallel FORS trees. The n -bit message digest is split into k indices, each signed by one FORS tree.

SK.seed	n bytes
SK.prf	n bytes
PK.seed	n bytes
PK.root	n bytes

Fig. 7: SPHINCS+ private key

PK.seed	n bytes
PK.root	n bytes

Fig. 8: SPHINCS+ public key

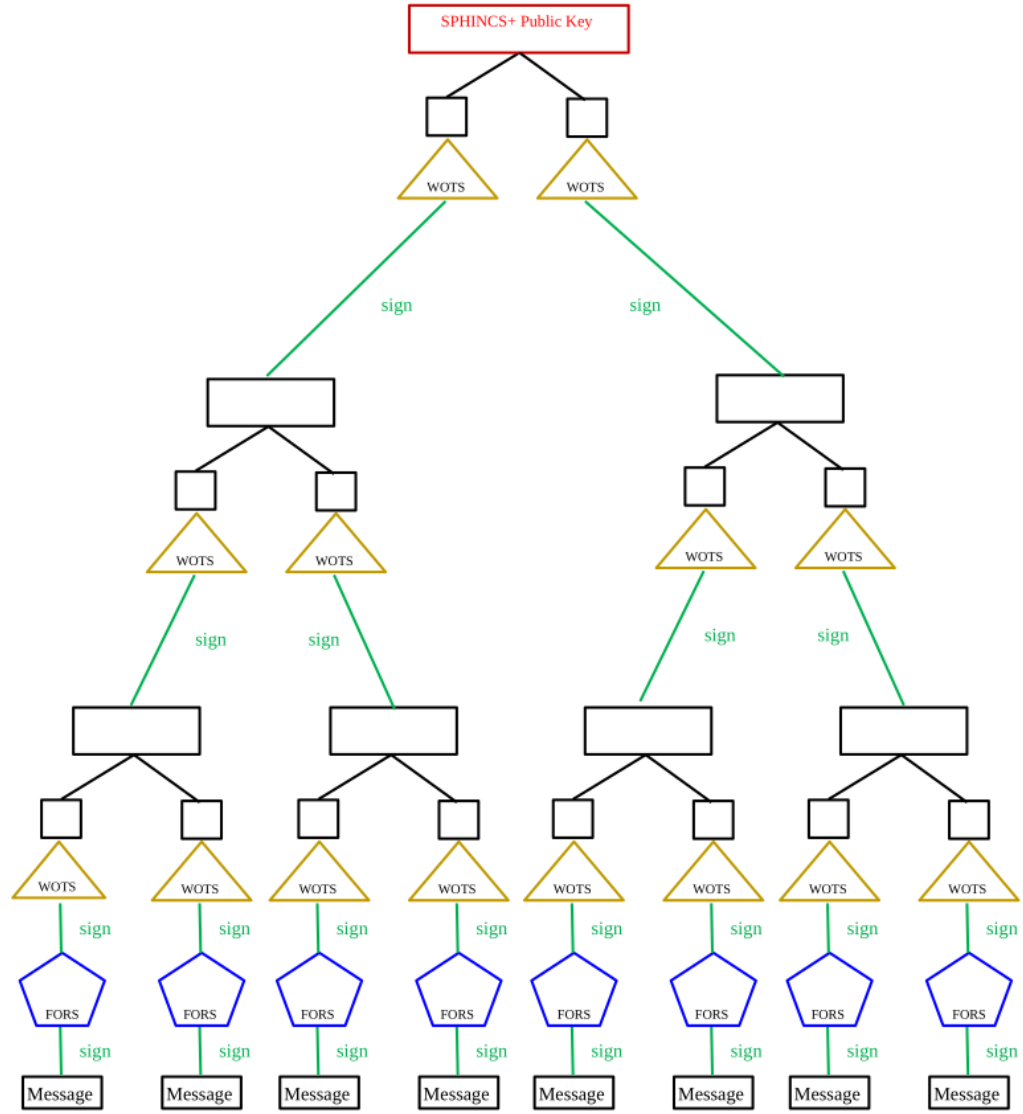


Fig. 9: SPHINCS+ General structure

4.2 SPHINCS+ Key Generation

Key generation derives a full SPHINCS+ key pair from three seeds: the secret seed, the PRF key, and the public seed. It first initializes the ADRS value with a zero-filled 32-byte string and configures it for the top-level XMSS tree by setting the layer address to $d - 1$. The root node of that tree is computed via a single call to `xmss_node`, which absorbs the secret seed, the public seed, and the ADRS state. Finally, the algorithm returns the secret key tuple ($SK.seed$, $SK.prf$, $PK.seed$, $PK.root$) alongside the public key ($PK.seed$, $PK.root$) (see Algorithm 5).

Algorithm 5 SPHINCS+_keyGen – Generate a SPHINCS+ key pair

Input: None

Output: SPHINCS+ private key SK , SPHINCS+ public key PK

```

1: Generate uniformly random  $n$ -byte strings:  $SK.SEED$ ,  $SK.PRF$ ,  $PK.SEED$ 
2: Initialize ADRS as a zeroed address structure
3:  $PK.root \leftarrow \text{treehash}(SK.SEED, PUB\_SEED, ADRS)$ 
4:  $SK \leftarrow SK.SEED \parallel SK.PRF \parallel PUB.SEED \parallel PK.root$ 
5:  $PK \leftarrow PK.SEED \parallel PK.root$ 
6: return  $SK$ ,  $PK$ 

```

4.3 SPHINCS+ Sign

Signature generation begins by initializing the ADRS to zero and optionally randomizing the message randomizer input. The PRF keyed by $SK.prf$ produces the randomness R that is prepended to the signature. A message digest is then computed over R , the public seed, the root, and the message itself. This digest is split into the message digest bits md and the tree/leaf indices idx_tree , idx_leaf . The FORS scheme is applied at the lowest layer (layer 0) to sign md , yielding SIG_FORS and an authenticated public key PK_FORS . Finally, the hypertree (HT) signature over PK_FORS is computed and appended, producing the full SPHINCS+ signature (see Algorithm 6)

Signature
Randomness R
FORS Signature SIG_{FORS}
HT Signature SIG_{HT}

Table 3: SPHINCS+ signature data format

Algorithm 6 `spx_sign` – Generating a SPHINCS+ signature

Input: Message M , private key $SK = (SK.seed, SK.prf, PK.seed, PK.root)$
Output: SPHINCS+ signature SIG

```

1:  $ADRS \leftarrow \text{toByte}(0, 32)$ 
2:  $opt \leftarrow PK.seed$ 
3: if RANDOMIZE then
4:    $opt \leftarrow \text{rand}(n)$ 
5: end if
6:  $R \leftarrow \text{PRF\_msg}(SK.prf, opt, M)$ 
7:  $SIG \leftarrow R$ 
8:  $digest \leftarrow H\_msg(R, PK.seed, PK.root, M)$ 
9:  $tmp\_md \leftarrow \text{first } \lfloor (ka + 7)/8 \rfloor \text{ bytes of } digest$ 
10:  $tmp\_idx\_tree \leftarrow \text{next } \lfloor (h - h/d + 7)/8 \rfloor \text{ bytes of } digest$ 
11:  $tmp\_idx\_leaf \leftarrow \text{next } \lfloor (h/d + 7)/8 \rfloor \text{ bytes of } digest$ 
12:  $md \leftarrow \text{first } ka \text{ bits of } tmp\_md$ 
13:  $idx\_tree \leftarrow \text{first } (h - h/d) \text{ bits of } tmp\_idx\_tree$ 
14:  $idx\_leaf \leftarrow \text{first } h/d \text{ bits of } tmp\_idx\_leaf$ 
15:  $ADRS.setLayerAddress(0)$ 
16:  $ADRS.setTreeAddress(idx\_tree)$ 
17:  $ADRS.setType(FORS\_TREE)$ 
18:  $ADRS.setKeyPairAddress(idx\_leaf)$ 
19:  $SIG\_FORS \leftarrow \text{fors\_sign}(md, SK.seed, PK.seed, ADRS)$ 
20:  $SIG \leftarrow SIG || SIG\_FORS$ 
21:  $PK\_FORS \leftarrow \text{fors\_pkFromSig}(SIG\_FORS, md, PK.seed, ADRS)$ 
22:  $ADRS.setType(TREE)$ 
23:  $SIG\_HT \leftarrow \text{ht\_sign}(PK\_FORS, SK.seed, PK.seed, idx\_tree, idx\_leaf)$ 
24:  $SIG \leftarrow SIG || SIG\_HT$ 
25: return  $SIG$ 

```

4.4 SPHINCS+ Verify

Verification mirrors signing: it parses the incoming signature into R , SIG_FORS , and SIG_HT . The verifier recomputes the message digest and splits it into md , idx_tree , and idx_leaf . It then re-derives the FORS public key by running $fors_pkFromSig$ on SIG_FORS and md . Finally, the hypertree signature is checked with ht_verify , which returns *true* only if every component matches the stored public root (see Algorithm 7).

Algorithm 7 spx_verify – Verify a SPHINCS+ signature SIG on a message M using a SPHINCS+ public key PK

Input: Message M , signature SIG , public key PK

Output: Boolean

```

1:  $ADRS \leftarrow \text{toByte}(0, 32)$ 
2:  $R \leftarrow SIG.\text{getR}()$ 
3:  $SIG\_FORS \leftarrow SIG.\text{getSIG\_FORS}()$ 
4:  $SIG\_HT \leftarrow SIG.\text{getSIG\_HT}()$ 
5:  $digest \leftarrow H\_msg(R, PK.seed, PK.root, M)$ 
6:  $tmp\_md \leftarrow \text{first } \lfloor (ka + 7)/8 \rfloor \text{ bytes of } digest$ 
7:  $tmp\_idx\_tree \leftarrow \text{next } \lfloor (h - h/d + 7)/8 \rfloor \text{ bytes of } digest$ 
8:  $tmp\_idx\_leaf \leftarrow \text{next } \lfloor (h/d + 7)/8 \rfloor \text{ bytes of } digest$ 
9:  $md \leftarrow \text{first } ka \text{ bits of } tmp\_md$ 
10:  $idx\_tree \leftarrow \text{first } (h - h/d) \text{ bits of } tmp\_idx\_tree$ 
11:  $idx\_leaf \leftarrow \text{first } h/d \text{ bits of } tmp\_idx\_leaf$ 
12:  $ADRS.\text{setLayerAddress}(0)$ 
13:  $ADRS.\text{setTreeAddress}(idx\_tree)$ 
14:  $ADRS.\text{setType}(FORS\_TREE)$ 
15:  $ADRS.\text{setKeyPairAddress}(idx\_leaf)$ 
16:  $PK\_FORS \leftarrow fors\_pkFromSig(SIG\_FORS, md, PK.seed, ADRS)$ 
17:  $ADRS.\text{setType}(TREE)$ 
18: return  $ht\_verify(PK\_FORS, SIG\_HT, PK.seed, idx\_tree, idx\_leaf, PK.root)$ 
```

5 Some Attacks on Hash Based Signatures

This section examines some attacks on hash-based signature algorithms.

5.1 Intermediate Value Guessing Attack

This attack was defined in the paper by Booth et al.[2]. In the document by Hülsing et al. [6], it is stated that the secret key values used in the WOTS+ signature must be generated in a uniformly distributed , but the method is left to the implementer. Assume that the WOTS+ secret keys (**osk**) are generated from the seed values $\mathbf{seed}_{\text{OTS},i}$ as follows:

$$\mathbf{osk} = (x_{i,1}, x_{i,2}, \dots, x_{i,l}) = (F(\mathbf{seed}_{\text{OTS},i}, 1), \dots, F(\mathbf{seed}_{\text{OTS},i}, l))$$

Under this assumption:

- The signatures of messages whose base- w representation includes zero values leak the corresponding indices of **osk**. For instance, if the base- w representation of message M is $BM = (b_1, 0, b_2)$, then the signature $\sigma_M = (Z_1, x_{i,2}, Z_3)$ reveals $x_{i,2}$. (Here, each Z value is derived by applying the hash algorithm to a secret key index w times.)
- The **osk** values used to sign different messages are generated from different seed values.
- Using a different seed for each WOTS+ secret key allows a multi-target attack, as described below.

In this attack, the adversary proceeds as follows:

- Executes the signing oracle to obtain multiple WOTS+ message-signature pairs.
- Filters out pairs where fewer than two **osk** values are revealed.
- Assumes that a signature has the form $\sigma = (Z_1, \dots, Z_l)$ and the message base- w representation is $BM = (b_1, \dots, b_l)$. The adversary identifies two indices k_1 and k_2 such that Z_1 and Z_2 correspond to x_{i,k_1} and x_{i,k_2} respectively. (Here, k_1 and k_2 are the positions where the base- w value is zero.)
- From this, the adversary learns:

$$x_{i,k_1} = F(\mathbf{seed}_{\text{OTS},i}, k_1), \quad x_{i,k_2} = F(\mathbf{seed}_{\text{OTS},i}, k_2)$$

- Using these values, the adversary constructs tables from message-signature pairs (assuming layer j is known):

$$(x_{k_1,k_2}, x_{k_2,i}, j), \quad (x_{k_3,k_4}, x_{k_4,i}, j)$$

- For each guessed $\mathbf{seed}_{\text{OTS},i}$, the adversary reconstructs a candidate **osk'** and checks whether the generated value matches existing entries in the table. If both x_{k_1} and x_{k_2} match, the guess for $\mathbf{seed}_{\text{OTS},i}$ is considered correct.

Assuming the adversary correctly guesses $\mathbf{seed}_{\text{OTS},i}$, let the message M_i have the following XMSSMT signature structure. To forge a valid signature, the adversary uses one of the following strategies:

Case 1: Seed Belongs to Layer 0 At layer 0, the signature is generated by signing $H(R \parallel \text{ROOT} \parallel i, M_i)$. Let M_j be the adversary's chosen message. The adversary computes $H(R \parallel \text{ROOT} \parallel i, M_j)$, signs it with the recovered secret key, and replaces the original signature value with this forged one. The authentication path values (**Auth**) remain unchanged, as they depend on the secret key used, not the signed message.

Case 2: Seed Belongs to Layer $j > 0$ Suppose the recovered seed belongs to a WOTS+ instance at layer $j > 0$. In this case, the WOTS+ signs the root of layer $j - 1$. In XMSSMT, signature verification is performed by computing the root at each layer and comparing the top-layer root with the public key. If the seed belongs to a layer $j > 0$, the adversary can forge the subtree below layer j for a chosen message, replacing the signature value at layer j while keeping the rest of the signature unchanged. Since the upper layers remain valid, the signature is accepted as legitimate.

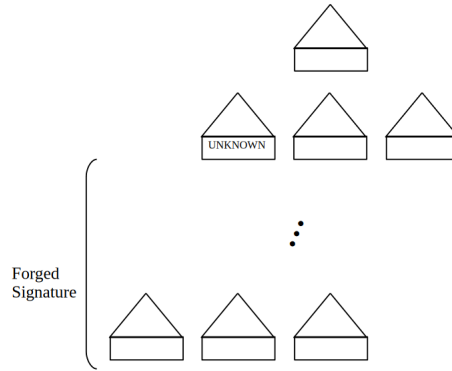


Fig. 10: Intermediate Value Guessing

5.2 Antonov's Attack

The attack described in [10] exploits the Merkle–Damgård construction of the SHA-256 algorithm. Assume there are six target messages $M_0, \dots, M_5$. For each message, different address values $\text{ADRS}_0, \dots, \text{ADRS}_5$ are used within the hash algorithm. At this stage, intermediate values of the compression function C are obtained for each address.

Then, by applying a collision attack, the adversary finds random values $x_0, \dots, x_5$, and captures pairwise collisions for the next set of intermediate values.

At this point, using the randomly found values $X_{0,1}, X_{2,3}, X_{4,5}$, a 3-branch collision is constructed.

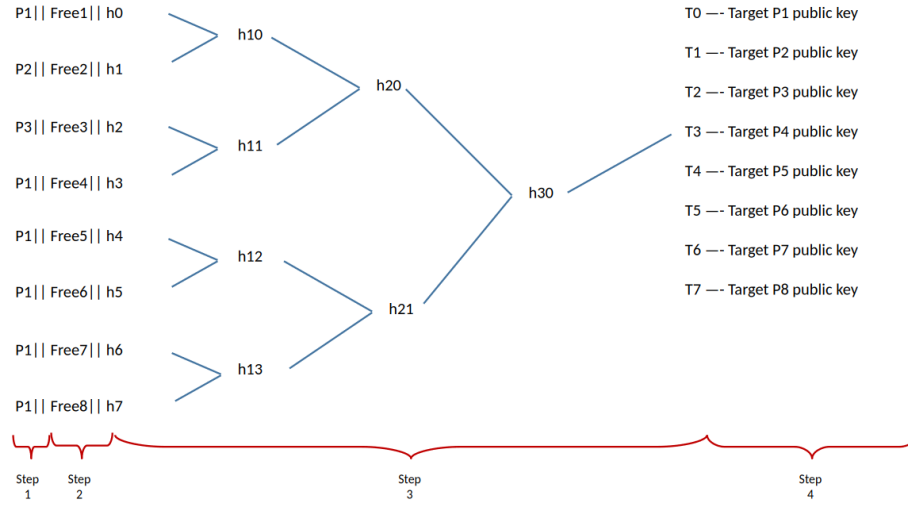


Fig. 11: Antonov's Attack

As the final step, the adversary finds a value Z such that the following equation holds for some $i \in \{0, \dots, 5\}$:

$$\text{SHA-256}(\text{ADRS}_i || x_i || X_{j,k} || Z) = \text{SHA-256}(\text{ADRS}_i || M_i)$$

For example, if the prediction is found for M_3 , then the equation

$$\text{SHA-256}(\text{ADRS}_3 || x_3 || X_{2,3} || Z) = \text{SHA-256}(\text{ADRS}_3 || M_3)$$

is satisfied, thus completing the multi-target prediction attack.

The complexity of finding a prediction in the final stage for t target messages is approximately $2^{256}/t$ compression function calls. The cost of finding an n -way collision is also determined by the number of C function evaluations.

This attack is broken down into four stages, as illustrated in Figure 11

Step 1: In this phase, the adversary collects a number of message-signature pairs signed using the same hash-based signature algorithm. Since the WOTS+ public keys used in the signature can be derived from the signature itself, the adversary also obtains these public keys. The attacker prefers messages with low checksum values at this stage (as lower checksums increase the probability of having F digits in the base- w representation). Although all message-signature pairs share the same PK.seed, each has a distinct ADRS. In Figure 11, $P_1, \dots, P_8$ represent the ADRS and PK.seed values. Finally, the attacker sorts these values.

Step 2: Here, the attacker selects random values $sk_1, \dots, sk_n$ and computes their hashes $2w$ times. By concatenating these values, they obtain the aggregated WOTS+ public keys denoted as $\text{free}_1, \dots, \text{free}_8$.

Step 3: The attacker appends random values h to the free values and uses the method explained at the beginning of this section to find an n -way collision. Unlike Stage 2, the attacker does not know the secret keys corresponding to the WOTS+ public keys in this step.

Step 4: In this stage, the attacker compares the found n -way collision value with the targeted public keys using the value h_{30} . $T_0, \dots, T_7$ represent the target public keys before the checksum is added. If a collision is detected, the attacker has successfully forged a new WOTS+ public key whose hash value matches that of a legitimate public key.

As a result of the above steps, the attacker can generate a forged signature with a fake key. In both SPHINCS+ and XMSS algorithms, public keys generated at each layer during verification are not checked directly—instead, their hash values are computed and passed on to the next layer. Therefore, even if the attacker’s keys differ from the legitimate keys, verification succeeds as long as the hash values match. The main constraint of this attack is that the message to be signed must follow a specific format:

```
XXXXXXXX XXXXXXXX XXXXXXXX FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF
                        FFFFFFFF
```

(where X can be any value). This requirement arises because the attacker knows the secret keys corresponding to the public keys used in Stage 2, but not those in Stage 3. In WOTS+ signatures, the public key is used to sign positions marked with F ; therefore, message segments where the secret key is unknown must be fixed to F to successfully carry out the attack.

5.3 Fault Injection Attacks

Fault injection attacks consist of scenarios in which one or more components of the algorithm are assumed to malfunction or operate incompletely due to external interference. For hash-based signature schemes, an example of a fault injection attack is described in the [11].

The WOTS+ one-time signature algorithm divides the message to be signed into w -bit chunks and generates the signature value by repeatedly applying the hash function to the secret key as many times as the numerical value of each chunk. At this stage, an attacker can compute the hash of the signature value one more time (when the message is not F) to create a valid signature. To prevent this, a checksum value is appended to the message before signing. This checksum is calculated as $c = (w - 1 - m)$, where m is the number of w -bit message chunks.

With this checksum, if the attacker computes extra hashes from the signature value, the checksum value decreases. Since the hash functions are designed to be resistant to preimage attacks, the attacker cannot compute the new checksum value. The attack described in [11] is based on the checksum calculation process malfunctioning due to an injected fault. If the checksum calculation does not work correctly, the attacker may generate a valid signature for a message of their choice.

5.4 Multi-Target Attacks

Multi-target attacks aim to find preimages when multiple output values of the used hash function are known. For a hash function with an n -bit output, the complexity of finding a preimage is 2^n . In multi-target attacks, if d different hash outputs are known, the complexity of finding a preimage reduces to $2^n/d$.

The XMSS and SPHINCS+ algorithms mitigate this situation by using keyed hash functions. In each usage of the hash function, different key and mask values are used, effectively differentiating the hash function for each operation and ensuring security against multi-target attacks.

6 Conclusion

Hash-based signature schemes provide a mature and theoretically sound foundation for post-quantum digital signatures. Their security is grounded in the robustness of cryptographic hash functions, making them well-suited for a future where quantum adversaries may render traditional public-key cryptosystems obsolete.

In this paper, we presented a comprehensive overview of hash-based signature constructions, starting from early schemes like Lamport and Merkle signatures to standardized schemes such as XMSS and the stateless SPHINCS+. We analyzed their structural differences, key components, and operational procedures, including signature generation, verification, and the role of one-time signatures like WOTS and FORS. We also discussed several cryptanalytic threats specific to HBS schemes, including multi-target attacks, intermediate value recovery, and fault injection attacks, emphasizing the need for implementation-level countermeasures, especially in embedded environments.

Future work may focus on improving efficiency, reducing signature sizes, and designing hybrid systems that combine the strengths of HBS with other post-quantum approaches to achieve broader applicability and resilience.

References

- [1] Daniel J Bernstein et al. “The SPHINCS+ signature framework”. In: *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*. 2019, pp. 2129–2146.
- [2] Roland Booth et al. “An Intermediate Secret-Guessing Attack on Hash-Based Signatures”. In: *International Workshop on Security*. Springer. 2021, pp. 195–215.
- [3] Gilles Brassard, Peter Høyer, and Alain Tapp. “Quantum cryptanalysis of hash and claw-free functions: Invited paper”. In: *LATIN’98: Theoretical Informatics*. Springer Berlin Heidelberg, 1998, 163–169. ISBN: 9783540697152. DOI: 10.1007/bfb0054319. URL: <http://dx.doi.org/10.1007/BFb0054319>.
- [4] Johannes Buchmann et al. “On the security of the Winternitz one-time signature scheme”. In: *International Journal of Applied Cryptography* 3.1 (2013), pp. 84–96.
- [5] David A Cooper et al. “Recommendation for stateful hash-based signature schemes”. In: *NIST Special Publication* 800.208 (2020), pp. 800–208.
- [6] A Huelising et al. *Rfc 8391: Xmss: extended merkle signature scheme*. 2018.
- [7] Andreas Hülsing. *WOTS+ – Shorter Signatures for Hash-Based Signature Schemes*. Cryptology ePrint Archive, Paper 2017/965. 2017. DOI: 10.1007/978-3-642-38553-7. URL: <https://eprint.iacr.org/2017/965>.
- [8] Leslie Lamport. *Constructing Digital Signatures from a One Way Function*. Tech. rep. CSL-98. This paper was published by IEEE in the Proceedings of HICSS-43 in January, 2010. Oct. 1979. URL: <https://www.microsoft.com/en-us/research/publication/constructing-digital-signatures-one-way-function/>.
- [9] Gaithersburg MD NIST. “Stateless Hash-Based Digital Signature Standard”. In: (2024). DOI: 10.6028/nist.fips.205.
- [10] Ray Perlner, John Kelsey, and David Cooper. “Breaking category five SPHINCS+ with SHA-256”. In: *International Conference on Post-Quantum Cryptography*. Springer. 2022, pp. 501–522.
- [11] Alexander Wagner et al. “Faulting Winternitz one-time signatures to forge LMS, XMSS, or signatures”. In: *International Conference on Post-Quantum Cryptography*. Springer. 2023, pp. 658–687.